

0.73.1. Glisser en une ligne : glisse_croix — de base, ailleurs

Le 0.73 avec le B, parti de (20, 20) pixels.

1. Ce qu'il faut comprendre

C'est le 0.73, avec le B, parti de (20, 20) pixels.

C'est la version de base : la deuxième place, seule. Le 0.73.2 met les deux ensemble.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1592 octets** de cartouche.

1. UNE VARIABLE — PX

```
uint8_t px = 20;    // le B, en pixels
```

2. UNE VARIABLE — PY

```
uint8_t py = 20;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

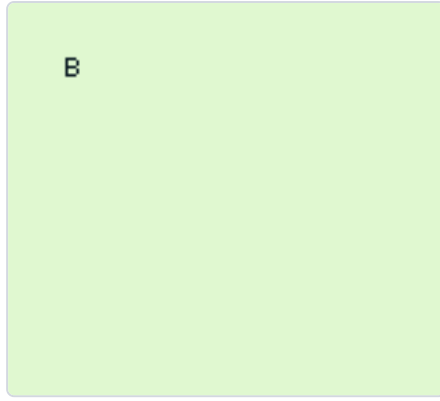
```
int main() {
    while (true) {
        image();
        glisse_croix(0, px, py, ALPHABET[1], 1);
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un B qui suit la croix, depuis le haut à gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le lutin 0 a glissé de 30 pixels

Ensuite : **0.73.2. Glisser en une ligne : glisse_croix — doublé, deux positions** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++